Prepared for BlackRock

What Aladdin Studio does, read from its public code

Aladdin Studio research · Version V8

Prepared by **Jacob Nikolau** | As of October 18, 2026 · V8 revision October 19, 2026

Part 1
Start here
The summary, and Aladdin in plain terms
Page 3

Part 2
Conclusions
The evidence and reasoning for each of the nine conclusions
Page 9

Part 3
Reference
Glossary, limits, method, sources and the verification notes for V8
Page 23

Independent analysis of public package files. It was not produced, reviewed or endorsed by BlackRock.

Contents

Nine conclusions about BlackRock's Aladdin Studio, drawn from the free software BlackRock publishes for it. Each one separates what the files state from what is inferred.

	Part 1. Start here The summary, and Aladdin in plain terms	**3**
	Summary	4
	Aladdin and Studio in plain terms	5
	How Aladdin and Studio fit together	6
	What the packages contain	7
	Part 2. Conclusions The evidence and reasoning for each of the nine conclusions	**9**
1	Studio is a place inside Aladdin where client teams write, run and store their own code	10
2	Code can reach Aladdin in two ways, and large data pulls likely go through Data Cloud	11
3	Clients can load their own data into Aladdin and use it in Aladdin's portfolio tools	12
4	Code can take real actions in Aladdin, including approvals and cash payments	14
5	Studio opens up the Aladdin apps clients already use, from research to accounting	15
6	Much of Studio is built for jobs that run on their own and alert people	17
7	Studio serves two kinds of users: business staff who write Python, and IT teams that connect systems	18
8	One central BlackRock team sets the design standard for the APIs, and the set is built to grow	19
9	The public APIs are a small, rarely updated part of a larger set that only clients can see	21
	Part 3. Reference Glossary, limits, method, sources and the verification notes for V8	**23**
	Terms used in this report	24
	What the files cannot tell us	26
	Method and sources	26
	Sources	27
	Verification notes for V8	33

How to read the labels

Each conclusion separates what the files state from what is inferred. A black square marks a **Stated** claim: the package files, or PyPI's public records about them, say this directly. An orange square marks an **Inference**, rated Strong, Moderate or Tentative by how firmly the evidence supports it. Small S tags, such as S1, point to the sources sheet in the Reference part.

■ **Stated**	The package files, or PyPI's public records about them, say this directly.
■ **Inference · Moderate**	The clues fit, but another explanation is still possible.
■ **Inference · Strong**	Several separate clues point the same way, and nothing in the files points the other way.
■ **Inference · Tentative**	The conclusion rests on one clue, or on something the files leave out.

Part 1

Start here

The summary, and Aladdin in plain terms

Summary	4
Aladdin and Studio in plain terms	5
How Aladdin and Studio fit together	6
What the packages contain	7

Summary

Aladdin is BlackRock's software for running investment portfolios. Aladdin Studio lets a client's own staff write programs that read Aladdin's data and take real actions in it. The free software BlackRock publishes for Studio shows this in detail.

58

APIs, or doorways for code, in the public files, with 326 operations between them

50%

Share of operations that change something in Aladdin rather than read or check it (162 of 326)

9

Areas of Aladdin the add-on APIs reach, from research to accounting, grouped from 11 add-ons

3

Operations that create, change or cancel a cash payment and send the payment instruction

Source: AladdinSDK core package and 11 API add-ons on PyPI, retrieved October 18, 2026; analysis of the 58 API definition files inside them. Totals that include the core package are this report's own tally; see "What the packages contain."

Each conclusion pairs facts stated in the files, such as the fact that code can place orders and send cash payments, with an inference drawn from them. Every inference is flagged, rated for confidence and explained step by step. The next two sections explain Aladdin, the key terms, the flags and how Studio fits in.

Exhibit 1. Nine conclusions, and how firmly each one is supported

#	Conclusion	Stated	Inference
1	Studio is a place inside Aladdin where client teams write, run and store their own code	1 claim	Strong
2	Code can reach Aladdin in two ways, and large data pulls likely go through Data Cloud	1 claim	Tentative
3	Clients can load their own data into Aladdin and use it in Aladdin's portfolio tools	1 claim	Moderate
4	Code can take real actions in Aladdin, including approvals and cash payments	2 claims	Strong
5	Studio opens up the Aladdin apps clients already use, from research to accounting	1 claim	Strong
6	Much of Studio is built for jobs that run on their own and alert people	1 claim	Strong

#	Conclusion	Stated	Inference
7	Studio serves two kinds of users: business staff who write Python, and IT teams that connect systems	2 claims	Moderate
8	One central BlackRock team sets the design standard for the APIs, and the set is built to grow	2 claims	Strong
9	The public APIs are a small, rarely updated part of a larger set that only clients can see	1 claim	Moderate; Tentative

Source: This report. "Stated" marks what the package files, or PyPI's records about them, state. "Inference" marks inferences, rated by confidence as explained under "How to read the flags."

Aladdin and Studio in plain terms

What Aladdin is	Aladdin is BlackRock's software for running investment portfolios. Asset managers, insurers, pension funds and other investors use it to research investments, decide what to buy and sell, place trades, check risk, and handle the accounting and paperwork after each trade. S75
What Studio is	Aladdin Studio is the part of Aladdin where a client's own staff build software on top of it. BlackRock shows little of the Studio screens and catalog in public. The files point to a developer guide on each client's own Aladdin website, where users log in. S67 S68 What BlackRock does show is a Studio product page, a Studio for Aladdin Risk page, an APIs page that describes the portal's catalog, Data Dictionary, notebooks and code samples, a 27-minute Snowflake podcast with Studio's head of engineering, a podcast with Studio's general manager, the engineering blog post that announced the SDK, and job postings for Studio Developer. S91 S94 – S98 S103 None of those shows the screens or lists the APIs, which is why this report reads the package files.
How BlackRock describes it	Dave Woodhead, head of engineering for Aladdin Studio, gives Studio three pillars. "Aladdin Studio is essentially the user experience that allows people to think of Aladdin as a platform. It exposes everything Aladdin knows how to do as a set of APIs events." Second, "The Aladdin data cloud built on top of Snowflake takes all of that information that's generated through our analytics factory and inside of Aladdin and pushes that into Snowflake." Third, "increasingly we're building a set of developer tools," so that builders "have sandboxes, research environments, they can start code, they can build applications. They can automate their workflows." On scale, inside BlackRock, "we have 3,000 citizen developers," and "I think that number has probably peaked," as natural-language interfaces take over from Python and R. The figure is BlackRock's own staff, not clients. S91
Where the evidence comes from	BlackRock publishes one part of Studio openly: a free toolkit called AladdinSDK, written in the Python programming language. Programmers download it from PyPI, the public website for Python software. The toolkit comes as 12 packages: a core package and 11 add-ons. Each package includes files that describe the APIs it contains, and the core package includes long instructions. Reading those files is the closest an outsider can get to seeing the API surface Studio exposes. They are the best public view of the APIs, not of Studio as a whole. S79 – S90
Hosting and people	Aladdin runs on Microsoft Azure today, with an AWS option announced in December 2025 for the second half of 2026. The Aladdin organization is about 7,000 people, more than 4,000 of them engineers, serving hundreds of clients in 70 countries. The public voices on Studio are Woodhead (engineering), Richard Newman (general manager, Aladdin Studio), Lance Braunstein (head of Aladdin Engineering), Sudhir Nair (Global Head of Aladdin) and Jeff Miller (Global Head of Data Platform and Product Engineering, Chief Architect). S91 S93 S94 S99 S100

Four terms used throughout

Term	Meaning
API	A doorway that lets one program ask another for data or tell it to do something. Aladdin's APIs let a client's code work with Aladdin.
Operation	One specific request an API accepts, such as "cancel an order" or "find watchlists." The public APIs have 326.
Toolkit	AladdinSDK, the free Python software that programmers install to work with Aladdin from their own code. It comes as a core package plus 11 add-ons that hold most of the APIs.
Data Cloud	Aladdin's data warehouse, where a client's Aladdin data is stored as tables. It runs on Snowflake, a cloud data company. S9 S76

How to read the flags

An inference is a conclusion drawn from the evidence. The files do not say it outright. Each one carries a rating for how firmly the evidence supports it.

Flag	Meaning
■ **STATED**	The package files, or PyPI's public records about them, say this directly. The evidence lists give the exact words in small print. Counts are this report's own tallies of the files.
■ **INFERENCE · STRONG**	Several separate clues point the same way, and nothing in the files points the other way.
■ **INFERENCE · MODERATE**	The clues fit, but another explanation is still possible.
■ **INFERENCE · TENTATIVE**	The conclusion rests on one clue, or on something the files leave out.

Some exhibits in the original layout also use yellow to highlight names, shade the Studio box or mark one category in a chart. Only a label that reads Inference marks an inference.

Other terms are explained where they first appear, and in the glossary.

How Aladdin and Studio fit together

Investment teams work in Aladdin applications by hand. Studio builders write programs that reach the same applications and data through code. The lower half follows one Studio job.

Exhibit 2. Aladdin, Studio and the people who use them

	Investment teams	Studio builders ■ Inference
Who	Portfolio managers, traders, and risk, operations and accounting staff	Analysts and operations staff who write Python, and IT teams that connect systems
How	work by hand	write and run programs
Where	**Aladdin applications**: applications and screens named in the API files. Aladdin Dashboard, Order Monitor, Events Monitor, Explore, Risk Radar, CashMaster, CID Manager, FX Center, DECO	**Aladdin Studio** ■ Inference: a hosted workspace for building on Aladdin. Studio UI, Studio Project, Studio Space, Studio Compute, S3 Storage Integration, API Applications, API Token, Studio Notifications, ADC Admin Center

Add-on APIs: 301 operations to read data and take actions.

Where the APIs reach, across the life of an investment

Research	Portfolio management	Trading	Risk and compliance	Operations	Accounting
	CashMaster, CID Manager, FX Center	Order Monitor	Risk Radar, Explore	DECO, Events Monitor	

SQL: whole tables. **Aladdin Data Cloud (ADC)**: a client's Aladdin data, stored as tables. Runs on Snowflake. Clients can also load their own data here and into the portfolio tools. ■ Inference: programs likely pull large data sets from here.

One Studio job, step by step. **An illustration built from features named in the files.**

Step	What happens	Flag
1	A builder writes a Python program with AladdinSDK.	■ Stated
2	The program runs on a schedule in Studio Compute.	■ Inference
3	It pulls a table of the portfolio's data from Aladdin Data Cloud with SQL.	■ Inference
4	It starts the portfolio's risk checks in Risk Radar through an API.	■ Stated
5	If a rule is broken, a Studio Notification emails the team.	■ Stated

The files do not describe this exact job, but every step uses features they name. Steps 2 and 3 are flagged because the files do not say that Studio Compute runs scheduled jobs (conclusion 6) or that large data pulls go through Data Cloud (conclusion 2).

Source: AladdinSDK package files, which name every application and Studio feature shown (sources on Exhibits 4 and 8); BlackRock's public description of Aladdin. Yellow chips reading Inference in the original layout mark inferences; plain yellow labels are names from the API files. **S75**

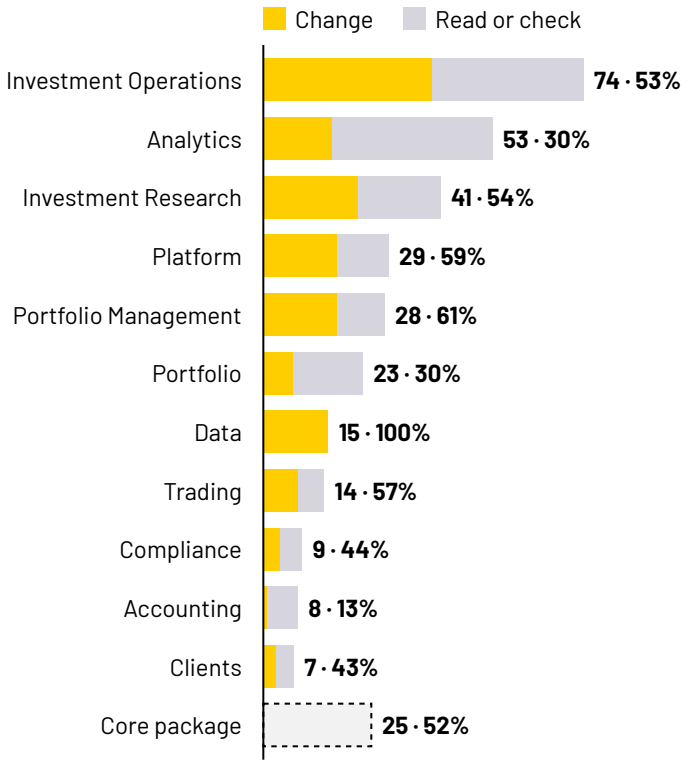
What the packages contain

The core package holds the toolkit's own code and 4 APIs: one for signing in, two for Studio Notifications, and a demo. The other 11 packages are add-ons that hold 54 APIs, each covering a business function, such as trading or accounting. Each API comes with a definition file that lists every operation it offers, 326 in all.

A note on the counts. The add-on figures in this report (54 APIs and 301 operations; 24 labeled alpha, 6 beta, 23 stable and 1 sunset) have been independently recomputed from the public packages and match. The core package's 4 APIs and 25 operations, and every total that includes them (58 APIs, 326 operations, 27 alpha and 7 beta, 218 operations with access tags, 65 batch operations, 11 background-job operations and 90 design-checker notes) are this report's own tally of the core package and have not been independently checked. They are arithmetically consistent with the add-on figures.

Exhibit 3 sorts the operations into three groups. Read operations look something up. Check operations validate data or run risk rules, screening criteria or a demo. Change operations create, update, approve, cancel or send something. Investment Operations, the add-on that handles the work after a trade, is the largest, and more than half its operations change something.

Exhibit 3. Operations in each package, by what they do



Package	All operations	Share that change
Investment Operations	**74**	**53%**
Analytics	53	30%
Investment Research	41	54%
Platform	29	59%
Portfolio Management	28	61%
Portfolio	23	30%
Data	15	100%
Trading	14	57%
Compliance	9	44%
Accounting	8	13%
Clients	7	43%
Core package	25	52%

Source: AladdinSDK 2.0.0b10 and the 11 API add-ons 2.0.0b3 on PyPI, retrieved October 18, 2026. In the original layout each bar is split into change, check and read. Operations are sorted by name, then by type of request; the rules are in the method section. **S79 – S90**

How a program signs in, from the files. Every add-on API definition names the same scheme. Each request carries an application key in a VND.com.blackrock.API-Key header, registered in Studio, plus one of: OAuth 2.0 through endpoints at client.blackrock.com, a client key in an X-API-Key header, or basic sign-in. OAuth scopes are set per resource, for example trading.order_management.order.v1.Order:read and :write; these are the "access tags" counted in the method section. Operations still cite Aladdin's own permission codes, so authorization has two layers: the scope at the door, the Aladdin permission inside. Data Cloud sign-in is separate: Snowflake OAuth, with the token fetched from Aladdin's TokenAPI where the client has set that up, or a Snowflake JWT with RSA keys. **S1 S7 S29 S30 S73 S101**

Part 2

Conclusions

The evidence and reasoning for each of the nine conclusions

1	Studio is a place inside Aladdin where client teams write, run and store their own code	10
2	Code can reach Aladdin in two ways, and large data pulls likely go through Data Cloud	11
3	Clients can load their own data into Aladdin and use it in Aladdin's portfolio tools	12
4	Code can take real actions in Aladdin, including approvals and cash payments	14
5	Studio opens up the Aladdin apps clients already use, from research to accounting	15
6	Much of Studio is built for jobs that run on their own and alert people	17
7	Studio serves two kinds of users: business staff who write Python, and IT teams that connect systems	18

1. Studio is a place inside Aladdin where client teams write, run and store their own code

WHY IT MATTERS Studio is more than a connection to Aladdin. It gives a client's team one place inside Aladdin to build programs, run them and keep the results.

■ **STATED** The files name nine Studio parts, all shown in Exhibit 4, including Studio Projects, Studio Compute and Studio Notifications.

■ **INFERENCE · STRONG** Together these parts make Studio a hosted workspace, not only a set of APIs.

Exhibit 4. The Studio parts named in the package files

Aladdin Studio ■ Inference: these parts form one hosted workspace

Part	What the files say it is
Studio Project	Personal or shared workspace that API Applications and storage belong to
Studio Compute	Computers inside Studio that run a client's code
Studio Space	Named in alert settings; not defined in the files
Studio Notifications	Email or Studio UI push alert to a user, Project or Space
S3 Storage Integration	File storage set up under a personal or shared Project
API Token	A key for a simpler kind of sign-in
Studio UI	Studio's own screens
ADC Admin Center	Studio's screen for managing Data Cloud datasets
API Applications	Where programs get their sign-in details

AladdinSDK, BlackRock's free Python toolkit, runs on a laptop or in Studio Compute. It reaches the **Aladdin APIs** (54 add-on APIs and 301 operations that reach Aladdin apps) and **Aladdin Data Cloud** (Aladdin's data warehouse, run on Snowflake).

Source: AladdinSDK core instructions, settings for where code runs, storage, access keys, sign-in and alerts; Studio notification API definitions; AdcDatasetAPI definition. Every box is named in the files. The yellow flag marks the inference. **S1 – S8**

■ What the evidence shows (STATED)

- Programs get their sign-in details from an API Application set up in a Studio Project. Core instructions: "by creating a OAuth application under a Project" **S1**
- Users can set up file storage, called an S3 Storage Integration, under a personal or shared Studio Project. Core instructions: "Create a Storage integration on Studio UI under a personal / shared project" **S2**
- Code can run on a person's laptop or in Studio Compute. Core instructions: "AladdinSDK can run on a developer's local machine or in Aladdin Studio Compute sessions." **S3**
- Studio Notifications go by email or Studio UI push to a user, Project or Space. One event type they cover is ACM, which this report reads as Aladdin Credit Management; that expansion is unverified, since only the value ACM appears in the files. Core instructions, alert settings; Studio notification API definitions **S4** **S5**

The reasoning (INFERENCE · STRONG)

A product that only offered APIs would need one thing from a client: a way to sign in. The files describe much more.

Studio Compute runs code. S3 Storage Integration keeps files. Studio Projects, personal or shared, hold a person's or a team's work. Studio Notifications tell people when something happens.

The parts are tied together. Storage and API Applications belong to a Studio Project. That is how a shared workspace for building software is organized.

Outside the files, BlackRock says the same. Woodhead's third pillar is developer tools: "sandboxes, research environments." The Studio for Aladdin Risk page says Studio's "centralized experience supports the building, configuration, and governance of Studio workflows, including access management and usage monitoring." The APIs page lists the portal's catalog, API-framework documentation, Data Dictionary, Jupyter notebooks and multi-language code samples, and lists Aladdin Data eXchange (ADX) and Aladdin Tools for Excel (ATX) as Studio resources. The core instructions name an "Aladdin Developer" notebook environment. S91 S95 S96

> **What would change this view.** If Studio Compute or storage were sold separately, Studio would be narrower than this. The files present both as part of Studio.

2. Code can reach Aladdin in two ways, and large data pulls likely go through Data Cloud

WHY IT MATTERS A Studio program has two routes into Aladdin: the APIs, and SQL queries on Data Cloud. Each route likely suits a different job.

STATED The toolkit has two routes to Aladdin data: the APIs, and SQL queries on Aladdin Data Cloud, Aladdin's data warehouse.

INFERENCE · TENTATIVE Large data pulls likely go through Data Cloud, while the APIs handle specific records and actions.

Exhibit 5. Two routes into Aladdin

Route	From	To	What moves
API route	Client's program, written in Python, run on a laptop or in Studio Compute, through the AladdinSDK toolkit	Aladdin apps: look up and act. Such as Risk Radar for risk and DECO for collateral. 54 add-on APIs, 301 operations	Specific records and actions. Results can be fetched page by page where an API supports it
Data route	Same program	Aladdin Data Cloud: pull and load whole tables. Aladdin's data warehouse, run on Snowflake. Example: a table of cash entries	Whole tables at once, using SQL queries. *Inference: large data pulls likely come this way*

Source: AladdinSDK core instructions and install requirements; API definition files. The yellow band in the original layout is an inference. S9 – S12 S76

What the evidence shows (STATED)

- The data route is built on Snowflake's own software. The instructions' example queries a table of cash entries. Core instructions: "a wrapper around snowflake-connector-python or snowflake-snowpark-python" S9 S10
- Where an API supports it, the toolkit fetches results in pages, with settings for how many pages and how many records per page. Core instructions: "maximum number of subsequent api calls made for pagination" S11
- The APIs do return some portfolio data. Valuations include portfolio and benchmark returns, accounting lots are listed by portfolio, and orders, trades and collateral statements carry prices. ValuationsAPI, AborLotAPI, OrderAPI, TradeAPI and CollateralStatementAPI data definitions S13 - S18

The reasoning (INFERENCE · TENTATIVE)

SQL is the usual way to pull a lot of data at once, and once Data Cloud access is set up, a program gets a direct SQL line into Aladdin's data warehouse.

The API route is built around specific records and actions, which suits specific items more than a firm's full data.

The APIs still return some portfolio data, such as valuations and accounting lots, so this is a likely pattern, not a rule.

Data Cloud, outside the files. Woodhead dates the start to "around 2019"; BlackRock launched it in 2021. It runs on Snowflake with Fivetran moving the data: "Fivetran allows us to take data from a single source and move that with low latency into a Snowflake landing zone." BlackRock shares data to "hundreds of Snowflake accounts" and manages "tens of thousands of pipelines." Jeff Miller says it replaced the old delivery: "We sent flat files, we sent large schema definitions, FTP and related transfers." Newman confirms "Aladdin data cloud specifically comes in, which is part of Studio." S91 S92 S93 S94 S102

> **What would change this view.** If clients mainly pull data through the APIs, the Data Cloud route matters less.

3. Clients can load their own data into Aladdin and use it in Aladdin's portfolio tools

WHY IT MATTERS Data can flow into Aladdin, not only out of it. A firm's own data, such as its research, could be used in Aladdin's portfolio management tools.

STATED Users with Studio admin rights can register custom datasets in Data Cloud, and custom investment data can be loaded for use in Aladdin's portfolio management tools.

INFERENCE · MODERATE This is meant for firms that want their own research or data inside the Aladdin screens their teams already use.

Exhibit 6. Three ways a client's own data gets into Aladdin

Route	What it does	Operations	Source
Load a table from code	Sends a table of data from a Python program straight into Data Cloud	Toolkit feature	Core instructions
Register a dataset in Data Cloud	Registers a "custom or non-standard" dataset and keeps its list of columns up to date. Needs the StudioAdmin permission type and the DATACLOUD permission group. Four operations, including release, are marked for internal BlackRock use	11	AdcDatasetAPI
Load custom investment data	Sets up a dataset, loads records in batches and switches it on or off, "for usage across Portfolio Management tools." File upload is kept for the CID Manager screen	17 (10 in the writer API, 7 in the reader API)	CIDWriterAPI, CIDReaderAPI

Source: AladdinSDK core instructions; AdcDatasetAPI, CIDWriterAPI and CIDReaderAPI definitions in the Platform and Portfolio Management add-ons. **S19 – S25**

What the evidence shows (STATED)

- The custom dataset feature brings custom investment data into Aladdin's portfolio management tools. Custom dataset API definitions: "CIDS provides a way to ingest the custom investment data into Aladdin for usage across Portfolio Management tools." **S20**
- BlackRock does not change the data, and whoever loads it owns it. Custom dataset API definitions: "CIDS does not transform the input data in any kind. The writer of the data owns it" **S21**
- Registering a dataset needs the StudioAdmin permission and the DATACLOUD permission group, and the dataset can be custom or non-standard. AdcDatasetAPI, register operation: "requires "StudioAdmin" Aladdin perm assigned to the user"; column refresh: the "StudioAdmin" Aladdin Permission Type and "DATACLOUD" Aladdin Permission Group **S22** **S23**
- Code loads records in batches. A separate file upload is kept for the CID Manager screen. CIDWriterAPI: "This endpoint is not meant for direct usage by other apps" **S25**

The reasoning (INFERENCE · MODERATE)

Most data products let clients take data out. These files also describe putting data in, and they say where it ends up: in Aladdin's portfolio management tools.

BlackRock does not change the data, and whoever loads it owns it. So users are expected to load data BlackRock does not check, such as a firm's own research scores.

The files say what the feature does. Who uses it, and for which data, is the inference. BlackRock has said it publicly: in 2024 Woodhead described "enabling clients to bring their own data into ADC, and to distribute Aladdin data to their own data ecosystems." **S92**

> **What would change this view.** If BlackRock's own teams use these routes mainly to load data from outside vendors, the client angle is weaker than it looks here.

4. Code can take real actions in Aladdin, including approvals and cash payments

WHY IT MATTERS Studio programs can do the work, not only report on it, including moving real orders and real money, with approvals and permissions built in.

■ **STATED** The operations include placing and cancelling orders and trades, approving risk rules and sending cash payments.

■ **STATED** A cash payment that sends money out needs the names of who confirmed and who reviewed it. Capital flow actions need Aladdin's newcash permissions, such as CASH_PAY.

■ **INFERENCE · STRONG** A Studio program can carry out steps of a firm's daily work that people would otherwise do by hand.

Exhibit 7. What the 162 change operations do, by this report's count

Type of change	Operations	Examples from the API files
Create	72	Place orders and trades. Set up new securities, from shares to swaps. Create research notes and users.
Update	43	Edit broker details, risk rules and strategies. Transfer analyst coverage.
Remove or cancel	36	Cancel orders and trades. Delete watchlists. Retire old dataset records.
Approve or move forward	8	Approve risk rules. Move cash movements and risk reviews to the next step. Pick corporate action options.
Send cash out	**3**	**Create, change or cancel a cash payment and send the payment instruction.**

Source: All 58 API definition files. Each operation is sorted by its name, then by its type of request; the rules are in the method section. **S26 – S28 S30 S31**

■ What the evidence shows (STATED)

- Code can place and cancel orders. An order is a portfolio manager's instruction to the trading desk. OrderAPI: "Filter, post or cancel orders. An order is a directive from a portfolio manager to the trading desk" **S26**
- Code can set up new securities, from credit default swaps to currency forwards. SecurityCreationAPI, 15 operations **S27**
- A cash operation sends money out, and needs the names of who confirmed and who reviewed the payment. MiscellaneousCashflowAPI: "sends money out through the transmission framework (confirmedBy and reviewedBy parameters are required" **S28**
- Capital flow actions need newcash permissions, such as CASH_PAY, and the API permission ALADDIN_API_USER. EnrichedCapitalFlowAPI: "standard API permissison ALADDIN_API_USER"; "standard newcash permissions" **S29 S30**

■ The reasoning (INFERENCE · STRONG)

A program that can only read data can produce reports. These APIs also let a program do the work: place an order, approve a rule, send a payment.

The controls point the same way. A field for who reviewed a payment only matters if code sends real payments. Capital flow actions check cash permissions such as CASH_PAY, so code works within set rights.

So a Studio program can carry out steps that people would otherwise do by hand.

> **What would change this view.** The files show what the APIs allow, not how often clients use it. The count also includes a demo API and 5 operations marked for internal use. S24 S32 S33

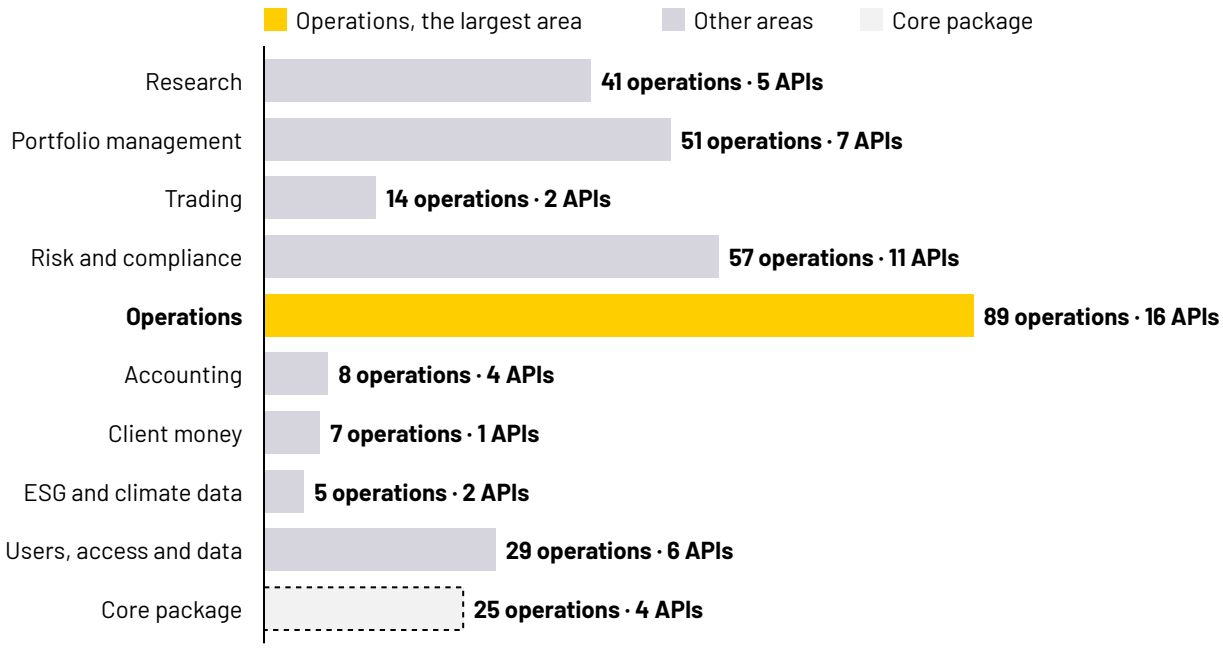
5. Studio opens up the Aladdin apps clients already use, from research to accounting

WHY IT MATTERS Code can work with what investment teams already see in Aladdin applications.

■ **STATED** The 11 add-ons reach from research to accounting, and several API files name the Aladdin application they connect to.

■ **INFERENCE · STRONG** Studio is a programmable door into the Aladdin a client already has.

Exhibit 8. The APIs cover Aladdin from research to accounting



The life of an investment, front to back:

Area	Operations	APIs	What the APIs cover	Applications and screens named
Research	41	5	Research notes, watchlists, analyst coverage, screening	
Portfolio management	51	7	Portfolios, cash by settlement date, custom datasets, strategies	CashMaster, CID Manager, FX Center
Trading	14	2	Orders, trades	Order Monitor

Area	Operations	APIs	What the APIs cover	Applications and screens named
Risk and compliance	57	11	Risk rules and exceptions; compliance rules and breaches	Risk Radar, Explore
Operations	**89**	**16**	**Corporate actions, cash payments, collateral, brokers, new securities**	**DECO, Events Monitor**
Accounting	8	4	Valuations, accounting lots, portfolio groupings	

Across the platform:

Area	Operations	APIs	What the APIs cover	Applications and screens named
Client money	7	1	Money into and out of funds	
ESG and climate data	5	2	ESG and climate risk data	
Users, access and data	29	6	Users, groups, permissions, datasets	ADC Admin Center
Core package	25	4	Sign-in, Studio Notifications, a demo	Studio UI

Source: All 58 API definition files. The nine areas group the 11 add-ons, as set out in the method section; the core package, in the dashed box in the original layout, is not one of them. In the original, bars are scaled to the largest area, Operations, and the black part shows operations that change something. Yellow labels are the applications and screens named in each box's API files. **S6 S8 S25 S34 – S40**

What the evidence shows (STATED)

- All seven risk APIs name Risk Radar, and four say they work on items as shown there. Risk API definitions: "as surfaced in Risk Radar" **S34**
- Collateral statements are built from agreements set up in DECO and can be viewed in the DECO tool. CollateralStatementAPI: "viewable in the DECO tool" **S35**
- An order API field shows up in Order Monitor, inside Aladdin Dashboard. OrderAPI: "visible in the "Basket" column in Order Monitor of Aladdin Dashboard" **S36**

The reasoning (INFERENCE · STRONG)

If Studio were a separate product, its APIs would describe their own data. Apart from the four core APIs and the Data Cloud dataset API, they describe things in Aladdin apps, at every stage of its work, if thinly in places such as trading. So Studio is a way to program the Aladdin a client already has.

> **What would change this view.** If more Studio-only data services sit behind the login, Studio is more of a product in itself.

6. Much of Studio is built for jobs that run on their own and alert people

WHY IT MATTERS The toolkit is built for scripts and scheduled jobs that run with nobody watching and then tell people what happened. That suits automating routine checks and updates.

- **STATED** The instructions mention "scripts / scheduled jobs," and the toolkit can email someone when a job fails.
- **INFERENCE · STRONG** Running routine work automatically is a main use of Studio, alongside one-off analysis.

Exhibit 9. Features for jobs that run with nobody watching

65	11	10	11
Operations named for handling many records in one request	Operations that check back on slow jobs running in the background	Operations for creating and finding Studio Notifications and managing subscriptions to them	Operations that validate data or run risk rules, screening criteria or a demo

Built into the toolkit, some switched on in settings: retries, limits on how fast requests are sent, a log file for each run, emails when a job fails, and Studio Notifications.

Source: All 58 API definition files; AladdinSDK core instructions, sections on retries, request limits, logging, alerts and error emails. **S4 S41 – S44**

What the evidence shows (STATED)

- The toolkit can email someone when a script or scheduled job fails. Core instructions: "while executing their scripts / scheduled jobs" **S41**
- It can keep a log file for every run, so there is a record of what happened. Core instructions: "Log files can also be created for each run to preserve run history." **S42**
- It can start slow jobs and check back on them later. Retries can be switched on in settings. Core instructions: "LRO Utilities to trigger and poll Long Running Operation endpoint calls" **S43 S44**
- Code can start all of a portfolio's risk checks at once. EvaluatorAnalyticsAPI: "Trigger all rules" **S45**

The reasoning (INFERENCE · STRONG)

A person working in a screen sees errors as they happen. A program that runs at 6 a.m. does not. Failure emails, run logs and retries are features for code that runs with nobody watching.

Batch operations fit the same picture, since they handle many records in one request. So do background jobs, which let slow work finish while the program checks back later. Add Studio Notifications, and the pattern is a job that runs on a schedule, checks or changes data, and tells people what happened.

Events, outside the files. The public packages hold no event stream beyond Studio Notifications, but BlackRock treats events as a channel in their own right. Woodhead names them in his first pillar: Studio "exposes everything Aladdin knows how to do as a set of APIs events." BlackRock's technical-writer postings for Studio describe streaming APIs and webhook subscriptions documented with AsyncAPI alongside OpenAPI. In the Build vs Buy webinar a BlackRock speaker says "you need access to APIs, you need access to events, and you need to make your workflow much more event based as opposed to batch based." So event-driven integration is part of what this conclusion covers, even though only the notification APIs are visible here. **S91 S97 S98**

What would change this view. The files do not show a scheduler inside Studio. Jobs may be scheduled with the client's own tools. That changes where the schedule lives, not the conclusion.

7. Studio serves two kinds of users: business staff who write Python, and IT teams that connect systems

WHY IT MATTERS Studio appears to serve both analysts who write some code and IT teams that connect whole systems. A message aimed at one group may miss the other.

■ **STATED** AladdinSDK is a Python toolkit. BlackRock's notes in nine risk API operations warn that a change "will break all citizen developers code."

■ **STATED** The toolkit also lets a firm route requests through its own API gateway, and lets programs sign in with system account credentials.

■ **INFERENCE · MODERATE** Day-to-day users include analysts and operations staff who write scripts. Client IT teams use the same APIs to connect Aladdin to other systems.

Exhibit 10. Evidence for two kinds of users

Signs of business staff who write code	Signs of IT teams connecting systems
BlackRock kept nine old web addresses so it would not "break all citizen developers code." A citizen developer is a business employee who writes code but is not a professional engineer.	A firm can send requests through its own API gateway, such as Microsoft's Azure API Management.
The instructions say settings help "citizen developers share code-snippets or projects."	Aladdin user accounts include "bot accounts granted access to Aladdin." The user API separates three kinds: "Aladdin Client's employees granted access to Aladdin," "BlackRock employees granted access to client's Aladdin products," and bots.
Results turn into tables in pandas, a standard Python tool for analysts, and save to Excel and CSV files.	Teams can build their own packages on top of AladdinSDK for "business specific" needs.
A step-by-step helper walks new users through setup.	Programs can sign in with "system account credentials."

Source: Risk API definitions; UserAPI definition; AladdinSDK core instructions, sections on settings, data handling, exports, the setup helper, sign-in, address rewriting and building on the toolkit. **S46 – S53 S77 S78**

■ What the evidence shows (STATED)

- BlackRock kept nine old web addresses unchanged so that citizen developers' code would keep working. Risk API definitions: "changing existing URI to suggested URI will break all citizen developers code" **S46**
- Firms can send requests through their own API gateway. Core instructions: "an enterprise API management platform such as Azure API Management" **S51**
- Aladdin user accounts can belong to bots as well as people. UserAPI: "bot accounts granted access to Aladdin" **S52**

The reasoning (INFERENCE · MODERATE)

BlackRock's engineers kept old web addresses so citizen developers' code would not break. So business staff already have code that depends on these APIs.

Tables in pandas, Excel files and a setup helper suit analysts more than full-time engineers. Gateways and system accounts are what IT teams use to connect systems.

So Studio serves both groups. The files do not show which group is larger. Woodhead's "3,000 citizen developers" are BlackRock's own staff, which supports the first group but says nothing about the client count. S91

> **What would change this view.** Some examples in the instructions point to BlackRock's own staff, such as an email address at blackrock.com. If most builders work at BlackRock, the client side is smaller than it looks. S54

8. One central BlackRock team sets the design standard for the APIs, and the set is built to grow

WHY IT MATTERS Once a programmer learns one Aladdin API, most of the others work the same way, and new APIs can be added easily.

- **STATED** Most packages share the same design patterns, and notes in all 12 packages show an automatic design checker at work.
- **STATED** New API add-ons plug in automatically, and the instructions point to a public project for building them.
- **INFERENCE · STRONG** A central team writes and enforces the standard, and expects many more APIs.

Exhibit 11. The same design patterns appear across the packages

Pattern used (number of operations); "–" means not used.

Package	Search actions	Many records at once	Background jobs	Remove or cancel	Design-checker notes	All operations
Investment Operations	18	24	1	7	17	74
Analytics	1	11	5	1	18	53
Investment Research	11	4	–	8	11	41
Platform	4	5	1	4	4	29
Portfolio Management	4	6	–	5	7	28
Portfolio	5	4	–	3	4	23
Data	–	–	–	–	14	15
Trading	3	4	2	4	8	14
Compliance	3	–	–	–	2	9
Accounting	3	–	1	–	3	8
Clients	2	2	–	–	1	7
Core package	3	5	1	4	1	25

Source: All 58 API definition files. Search actions counts operations named filter or search. Background jobs counts check-ins on slow jobs. Design-checker notes counts operations excused from a named rule. **S55** **S61** **S62**

What the evidence shows (STATED)

- 11 of 12 packages use the same ":filter" action, and 9 of 12 name batch operations with the same "batch" prefix. All 58 API definition files
- 90 operations carry notes excusing them from at least one design rule, with a reason. Each cites aip.dev, Google's public API design guide. Some rules carry the name "aladdin." Definition files in all 12 packages, for example "api-linter: aladdin::9016" **S55** **S61**
- Notes in the order API say one design choice "was agreed with agraph team" and that some names are "the agreed naming with Aladdin Graph team." OrderAPI definition; Core instructions calls them Aladdin Graph APIs **S56 - S58**
- New add-ons are picked up "without any additional setup steps." Core instructions; github.com/blackrock/aladdinsdk-plugin-builder **S59** **S60**

The reasoning (INFERENCE · STRONG)

A design checker leaves notes like these only when it is run routinely, and they appear in all 12 packages. The "aladdin" rules look like BlackRock's own additions to Google's, and the order API credits the Aladdin Graph team with agreeing its naming. That points to one central team, likely the Aladdin Graph team.

New APIs ship as add-ons that need no change to the core package, so the design expects the set to grow.

Other design signals in the add-on files, for technical readers: field names such as rpcStatus, pageToken, updateMask and enums ending UNSPECIFIED are the fingerprint of services written in protobuf and exposed through a gRPC gateway; 190 of the 301 add-on operations use POST; 35 operations carry an x-aladdin-lro flag that returns a background job (Exhibit 9's 11 are the check-back operations); records carry valid-from and valid-to dates, a bitemporal pattern; approvals follow four-eyes state machines; the Portfolio entity has about 850 attributes; fields named bqlExpression point to an internal query language. In the toolkit: settings through Dynaconf, secrets through the OS keyring, add-ons generated by openapi-generator through the plugin builder, and each add-on discovered through its api_registry.py and domain_apis_list.json files. S59 S60 S101

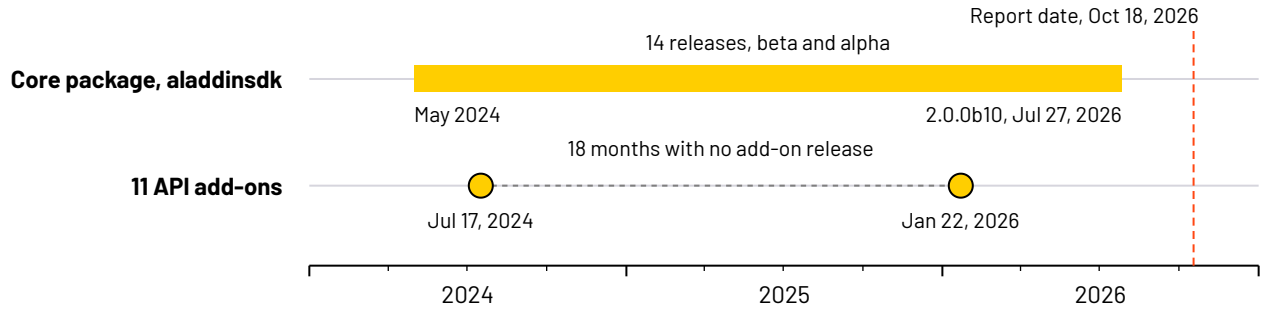
> **What would change this view.** If each business team applied the design rules on its own, with no shared team, this would weaken. The notes naming the Aladdin Graph team point the other way.

9. The public APIs are a small, rarely updated part of a larger set that only clients can see

WHY IT MATTERS Anyone judging Studio from the public files may see only part of what clients get. The core package is updated often, but the public API add-ons are not.

- **STATED** All 12 packages carry beta version labels, while the API files themselves label 23 of the 58 APIs stable.
- **INFERENCE · MODERATE** The public add-ons show a small part of the APIs clients can use.
- **INFERENCE · TENTATIVE** BlackRock copies its private APIs to the public add-ons only now and then.

Exhibit 12. The core package is released often; the API add-ons rarely are



Package	Releases on PyPI	Dates
Core package, aladdinsdk	14 releases, beta and alpha	First release May 2024; latest 2.0.0b10, Jul 27, 2026
11 API add-ons, released together each time	2 release dates	Jul 17, 2024 and Jan 22, 2026; 18 months with no add-on release

Report date: October 18, 2026.

Source: PyPI release records for aladdinsdk and the 11 API add-ons, retrieved October 18, 2026, as read by the author. Only the current version, 2.0.0b10 as of July 2026, has been independently verified; the release count and dates are the author's own reading of PyPI and are unverified. Each add-on date covers all 11 add-ons, released the same day. S63 S64 S69 – S71

What the evidence shows (STATED)

- PyPI's public records show 14 releases of the core package since May 2024, but add-on releases on only two dates: July 17, 2024 and January 22, 2026. PyPI release records, which are public records about the packages rather than the files themselves; author's tally, unverified S69 S70
- The instructions send readers to PyPI for the current list of APIs in each add-on. Core instructions: "refer documentation on PyPI for up-to-date list of APIs packaged in each plugin" S66
- Inside the API files, 27 of the 58 APIs are labeled alpha, 7 beta, 23 stable and 1 sunset, meaning it is being retired. For the 54 add-on APIs alone the independently recomputed split is 24 alpha, 6 beta, 23 stable and 1 sunset. API definition files, stability labels S65
- The developer guide and API calls live on each client's own Aladdin website, where users log in. UserAPI definition; core instructions S67 S68

The reasoning (INFERENCE · MODERATE)

Aladdin covers research, trading, risk, operations and accounting, yet the public add-ons hold 54 APIs, with only two for trading. So the full set of APIs is likely larger.

BlackRock's own words support this. The APIs page says the portal holds "dozens of APIs." Newman: "We're releasing soon the ability to actually access those analytic APIs, portfolio analysis, security analysis, things like that. That's all part of the Studio umbrella." The Studio for Aladdin Risk page offers to "Run analytics programmatically: Compute portfolio and security analytics for repeatable reporting, analysis, modeling, and integration into downstream systems." Positions, risk outputs, pricing and performance are absent from the public set, so those APIs sit behind the login. S94 S95 S96

The release pattern fits. The core package was released 14 times, while the add-ons were released on two dates, all 11 on the same day each time, even though most of their APIs are labeled alpha or beta. That looks like a periodic copy of a larger private set. This second inference rests on the release pattern alone, so it is rated tentative.

> **What would change this view.** If the full set is about the size of the public one, the public files show most of what clients get, and this conclusion is wrong.

Part 3

Reference

Glossary, limits, method, sources and the verification notes for V8

Terms used in this report	24
What the files cannot tell us	26
Method and sources	26
Sources	27
Verification notes for V8	33

Terms used in this report

Software terms

Term	Meaning
Add-on (plugin)	One of 11 extra packages that add APIs to the core package. The files call them plugins.
Alert	A message sent when something happens. Studio's alerts are called Studio Notifications. The toolkit can also email someone when a job fails.
Alpha, beta	Labels for test versions of software, before a stable release. Alpha is earlier than beta.
API gateway	A company's own system for managing and securing the APIs its software uses.
Background job	A slow task that keeps running after it is started. The code checks back later for the result.
Batch operation	An operation that handles many records in one request.
Citizen developer	A business employee who writes code but is not a professional engineer.
Core package	The main AladdinSDK package, named aladdinsdk. It holds the toolkit's own code, its instructions and 4 APIs.
Definition file	A file that lists every operation an API offers, with its web address and a description.
Design checker	A tool that tests every API against written design rules.
pandas	A standard Python tool for working with tables of data.
Package	A bundle of software that programmers download and install.
PyPI	The public website where Python software is published and downloaded.
Python	A popular programming language, widely used by analysts and data teams.
SQL	The standard language for asking a database for data.

Investment terms

Term	Meaning
Capital flow	Money coming into a fund from investors, or going back out to them.
Collateral	Assets pledged to cover what one party may owe another, often on derivatives.
Compliance rule	A limit a portfolio must stay within, such as a cap on one company's shares. A breach is called a violation.
Corporate action	An event a company announces that affects its shares or bonds, such as a dividend or merger. When investors can choose an option, the choice is called an election.
Order, trade	An order is a portfolio manager's instruction to buy or sell. The trade is the deal that results.
Swaps	Contracts in which two parties exchange payments, used to manage risks such as a borrower defaulting or interest rates changing.
Valuation	The value of a portfolio's assets on a given date, used in accounting. Aladdin's valuations also include returns.

Studio names in the files

Term	Meaning
AladdinSDK	BlackRock's free Python toolkit for working with Aladdin from code. It comes as a core package and 11 add-ons.
ADC Admin Center	Studio's screen for managing datasets in Aladdin Data Cloud (ADC).
API Applications	Where programs are registered so they can sign in to Aladdin's APIs. The instructions call each one an OAuth application. Each carries the application key sent in the VND.com.blackrock.API-Key header.
API Token	A key registered in the Studio UI for a simpler kind of sign-in.
S3 Storage Integration	File storage that a user sets up in the Studio UI under a personal or shared Studio Project. S3 is a widely used type of cloud file storage.
Studio Compute	Computers inside Studio that run a client's code. The instructions say "Aladdin Studio Compute sessions"; BlackRock's marketing calls the product Aladdin Compute. Both names refer to the same hosted environment.
Studio Notifications	Studio's alerts, sent by email or as a push alert in the Studio UI to the users, Projects or Spaces that subscribe.
Studio Project	A workspace in Studio, personal or shared. API Applications and storage are set up inside it.
Studio Space	Named in Studio Notification settings, as a place alerts can go and as an event type. It is not defined in the files.
Studio UI	Studio's own screens. Storage and API Tokens are set up there, and push alerts appear there.

Aladdin names in the files

Term	Meaning
ACM	An event type Studio Notifications cover. This report reads it as Aladdin Credit Management; the files give only the value ACM, so the expansion is unverified.
agraph, Aladdin Graph	Names the files use for Aladdin's APIs. The instructions call them Aladdin Graph APIs, and order API notes credit the Aladdin Graph team, or agraph team, with agreeing designs and names.
Aladdin Dashboard	An Aladdin application whose screens include Order Monitor, for orders, and Events Monitor, for corporate action events.
CashMaster	The Aladdin application used to publish projected cash.
CID Manager	An Aladdin screen for loading custom investment datasets. The files call it the CID Manager UI.
DECO	The Aladdin application where collateral agreements are set up and collateral statements are viewed. The files call it the DECO tool.
Explore	An Aladdin screen the files call Explore UI. Some risk API settings are aligned with it.
FX Center	Named in portfolio settings that say whether an account makes and receives foreign exchange payments through it.
newcash	Aladdin's set of permissions for cash and fund money work, such as CASH_PAY.
Risk Radar	The Aladdin application where risk rules, their exceptions and related reviews are tracked.
Transmission framework	The files' name for the way Aladdin sends a payment instruction out.

What the files cannot tell us

The package files describe what the software can do. They leave out several things that matter for understanding Studio as a product.

Question	Why the files cannot answer it
What the Studio screens look like	The files describe settings and APIs. They do not describe the screens.
How many clients use Studio, and how much	Nothing in the files measures use. The notes about citizen developers' code show that some use exists, but not how much.
What Studio costs and how it is sold	No Studio pricing or packaging appears.
What a Studio Space is	It is not defined in the files. It appears only in alert settings, as a place alerts can go and as an event type.
Whether clients use the change operations day to day	The files show what is allowed, not what happens.
What Studio offers for AI	The words AI, artificial intelligence, language model and machine learning do not appear in any of the files. Copilot appears only inside the name of one portfolio setting, about a distribution program. Absence in the files is not absence in the product: BlackRock's Copilot page says Aladdin Copilot is "now available to all Aladdin clients" and "Will not give investment advice"; a 2025 BlackRock engineering talk says Copilot's tools map to existing production Aladdin APIs; and Woodhead's AI plan includes a builder's assistant, "a co-pilot to help me build." S91 S96 S101
Which APIs exist behind the client login	Only the public add-ons are visible. The APIs page says the portal holds "dozens of APIs," with API-framework documentation, a Data Dictionary, Jupyter notebooks and multi-language samples, and lists ADX and ATX as Studio resources. S95

Method and sources

What was read

The 12 AladdinSDK packages were downloaded from PyPI on October 18, 2026 and opened without being installed or run. All 58 API definition files were read in full, including every operation and its data definitions. The core package's instructions (its README file) and its list of required software were read in full. Release dates come from PyPI's public records. The counts for the 11 add-ons were independently recomputed from the public packages after V7; the core-package counts and the release history remain the author's own tally. S71 S79 – S90

How operations were sorted

Each operation was sorted by its name, and otherwise by its type of request. Operations named filter, search, retrieve or list, and other requests that only fetch data, count as read. Operations named validate, evaluate or run count as check. All others count as change, in five groups: create (create, add, register, import, upload, upsert, submit, post, and plain POST requests); update (update, edit, transfer, refresh, reconcile, undelete, and PATCH or PUT requests); remove or cancel (delete, deleteChildren, cancel, remove, expire, terminate); approve or move forward (approve, transition, execute, elect, activate, release); and send cash, the three operations whose names end in SendTransmission. Send cash is checked first, so those three count there even though their names also start with create, update or cancel. One violation update named "add" counts as update. The result is 153 read, 11 check and 162 change. The demo API has operations in both the change and check groups. Five operations are marked for internal use: four dataset operations and one that creates risk workflows in bulk. Both the demo and these five are included in the counts. S24 S32 S33

BlackRock's own access tags, which are the OAuth scopes each operation requires (for example trading.order_management.order.v1.Order:read or :write), mark 218 of the 326 operations as read or write. Of those 218, every one this report counts as read is tagged read, and every one it counts as change is tagged write, except one that removes datasets from a cache, which is tagged read. The check group mixes both tags. S73

How add-ons were grouped into areas

Exhibit 8 groups the 11 add-ons into nine areas. Research, Trading and Accounting match their add-ons. Portfolio management joins Portfolio Management and Portfolio. Risk and compliance joins Compliance and seven Analytics APIs, and ESG and climate data holds the other two Analytics APIs. Operations joins Investment Operations and the Data add-on's one API, which sets up new securities. Client money is Clients, and Users, access and data is Platform. The core package is shown separately.

How absences were checked

Claims that something is missing come from a word search of all text in the packages, including the API data definitions and the toolkit's own code. The words AI, artificial intelligence, language model and machine learning do not appear. Copilot appears only as the name of one portfolio setting, whose description concerns a distribution program and does not mention AI. No operation carries a deprecated flag, though the text of a few says they are being retired or should no longer be used. S24 S72 S74

Sources

Every source is listed on the sources sheet below, with a link to the exact file and line. The small S tags throughout the report point to it.

Sources

Each link opens the exact source. Links into the packages open PyPI's file viewer, inspector.pypi.io, at the quoted line of the published file. The S numbers match the source tags on the conclusion pages, in the exhibit notes and on the method page. Every package-file link was opened in a browser on October 18, 2026 and checked to land on a line containing the quoted text. Web pages were checked to load.

Studio is a hosted workspace (conclusion 1, Exhibits 2 and 4)

Tag	What it shows	Where
S1	Programs get sign-in details from an API Application under a Studio Project	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 292
S2	File storage is set up under a personal or shared Studio Project	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 466
S3	Code runs on a laptop or in Studio Compute	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 213
S4	Studio Notifications go by email or push to users, Projects or Spaces	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 218
S5	ACM event type (read by the author as Aladdin Credit Management; unverified); Studio Space and Studio Compute event types	aladdinsdk 2.0.0b10 · studio_subscription definition file · line 1537
S6	Studio UI push notifications	aladdinsdk 2.0.0b10 · studio_subscription definition file · line 1512
S7	API Token registered via Studio UI	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 123
S8	ADC Admin Center is a Studio screen for Data Cloud datasets	asdk_plugin_platform 2.0.0b3 · AdcDatasetAPI definition file · line 13

Two routes into Aladdin (conclusion 2, Exhibit 5)

Tag	What it shows	Where
S9	The Data Cloud client wraps Snowflake's Python connector and Snowpark	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 377
S10	The instructions' example queries a table of cash entries	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 394
S11	Paging settings for API results	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 162
S12	Snowflake libraries in the install requirements	aladdinsdk 2.0.0b10 · METADATA · line 23
S13	Valuations include portfolio returns	asdk_plugin_accounting 2.0.0b3 · ValuationsAPI definition file · line 903
S14	Valuations include benchmark returns	asdk_plugin_accounting 2.0.0b3 · ValuationsAPI definition file · line 937
S15	Accounting lots are retrieved by portfolio	asdk_plugin_accounting 2.0.0b3 · AborLotAPI definition file · line 46
S16	Orders carry a market price	asdk_plugin_trading 2.0.0b3 · OrderAPI definition file · line 2250
S17	Trades carry a price	asdk_plugin_trading 2.0.0b3 · TradeAPI definition file · line 2103
S18	Collateral statements carry prices	asdk_plugin_investment_operations 2.0.0b3 · CollateralStatementAPI definition file · line 1806

Client data loaded into Aladdin (conclusion 3, Exhibit 6)

Tag	What it shows	Where
S19	A table can be written from a Python program into Data Cloud	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 397
S20	Custom investment data is loaded for use across Portfolio Management tools	asdk_plugin_portfolio_management 2.0.0b3 · CIDWriterAPI definition file · line 13
S21	BlackRock does not transform the data, and the writer owns it	asdk_plugin_portfolio_management 2.0.0b3 · CIDWriterAPI definition file · line 13
S22	Datasets can be custom or non-standard	asdk_plugin_platform 2.0.0b3 · AdcDatasetAPI definition file · line 1689
S23	Registering a dataset needs the StudioAdmin permission; column refresh also names the DATACLOUD permission group	asdk_plugin_platform 2.0.0b3 · AdcDatasetAPI definition file · line 1689
S24	Some dataset operations are for internal BlackRock use only	asdk_plugin_platform 2.0.0b3 · AdcDatasetAPI definition file · line 291
S25	File upload is kept for the CID Manager screen	asdk_plugin_portfolio_management 2.0.0b3 · CIDWriterAPI definition file · line 2096

Real actions, approvals and payments (conclusion 4, Exhibit 7)

Tag	What it shows	Where
S26	Code can post and cancel orders; what an order is	asdk_plugin_trading 2.0.0b3 · OrderAPI definition file · line 13
S27	Security setup for swaps, FX and more	asdk_plugin_data 2.0.0b3 · SecurityCreationAPI definition file · line 13
S28	A cash payment is sent out and needs confirmedBy and reviewedBy	asdk_plugin_investment_operations 2.0.0b3 · MiscellaneousCashflowAPI definition file · line 933
S29	Capital flows need the ALADDIN_API_USER permission	asdk_plugin_clients 2.0.0b3 · EnrichedCapitalFlowAPI definition file · line 13
S30	Capital flow actions need newcash permissions such as CASH_PAY	asdk_plugin_clients 2.0.0b3 · EnrichedCapitalFlowAPI definition file · line 46
S31	Risk rules can be approved from code	asdk_plugin_analytics 2.0.0b3 · RiskRuleAPI definition file · line 3929
S32	The core package includes a demo API	aladdinsdk 2.0.0b10 · train_journey definition file · line 13
S33	A risk workflow operation marked for internal use only	asdk_plugin_analytics 2.0.0b3 · RiskWorkflowAPI definition file · line 763

The Aladdin applications the APIs connect to (conclusion 5, Exhibits 2 and 8)

Tag	What it shows	Where
S34	Risk APIs work on items as surfaced in Risk Radar	asdk_plugin_analytics 2.0.0b3 · RiskRuleAPI definition file · line 13
S35	Collateral statements are viewable in DECO	asdk_plugin_investment_operations 2.0.0b3 · CollateralStatementAPI definition file · line 13
S36	Order Monitor in Aladdin Dashboard	asdk_plugin_trading 2.0.0b3 · OrderAPI definition file · line 2244
S37	Events Monitor in Aladdin Dashboard	asdk_plugin_investment_operations 2.0.0b3 · CorporateActionAPI definition file · line 663
S38	CashMaster, the Aladdin application for publishing cash	asdk_plugin_portfolio_management 2.0.0b3 · CashLadderAPI definition file · line 641
S39	Explore UI	asdk_plugin_analytics 2.0.0b3 · EvaluatorAnalyticsAPI definition file · line 1311
S40	FX Center	asdk_plugin_portfolio 2.0.0b3 · PortfolioToolkitAPI definition file · line 5659

Jobs that run on their own (conclusion 6, Exhibit 9)

Tag	What it shows	Where
S41	Error emails for scripts and scheduled jobs	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 660
S42	A log file for each run	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 598
S43	Starting and checking back on long-running jobs	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 322
S44	Retries are switched on in settings	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 553
S45	Code can trigger all of a portfolio's risk rules	asdk_plugin_analytics 2.0.0b3 · EvaluatorAnalyticsAPI definition file · line 275

Two kinds of users (conclusion 7, Exhibit 10)

Tag	What it shows	Where
S46	Old web addresses kept so citizen developers' code would not break	asdk_plugin_analytics 2.0.0b3 · RiskRuleAPI definition file · line 2582
S47	Settings help citizen developers share code; system account credentials	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 84
S48	Results convert to pandas tables	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 578
S49	Results export to Excel files	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 589
S50	A step-by-step setup helper	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 282
S51	Routing through a firm's own API gateway	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 667
S52	Bot accounts as Aladdin users; client employees and BlackRock employees as user types	asdk_plugin_platform 2.0.0b3 · UserAPI definition file · line 13
S53	Teams can build business-specific packages	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 818
S54	An example recipient address at blackrock.com	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 644

One central design standard (conclusion 8, Exhibit 11)

Tag	What it shows	Where
S55	A design-checker note excusing an operation from an aladdin rule, citing aip.dev	asdk_plugin_analytics 2.0.0b3 · RiskConfigAPI definition file · line 963
S56	One order API design choice "was agreed with agraph team"	asdk_plugin_trading 2.0.0b3 · OrderAPI definition file · line 46
S57	Order API names are "the agreed naming with Aladdin Graph team"	asdk_plugin_trading 2.0.0b3 · OrderAPI definition file · line 488
S58	The instructions call them Aladdin Graph APIs	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 310
S59	New add-ons are picked up without extra setup	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 342
S60	BlackRock's public project for building add-ons	github.com · blackrock/aladdinsdk-plugin-builder
S61	The public API design guide the notes cite	google.aip.dev · API Improvement Proposals
S62	The checker's rule format, for example core::0136::http-uri-suffix	linter.aip.dev · rule 136 http-uri-suffix

Public versus private APIs (conclusion 9, Exhibit 12)

Tag	What it shows	Where
S63	The core package carries a beta version label	aladdinsdk 2.0.0b10 · METADATA · line 3
S64	Each add-on carries a beta version label (Trading shown)	asdk_plugin_trading 2.0.0b3 · METADATA · line 3
S65	API stability labels; Risk Tasks is labeled sunset	asdk_plugin_analytics 2.0.0b3 · RiskTaskAPI definition file · line 17
S66	The instructions send readers to PyPI for the current list of APIs	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 346
S67	The developer guide lives on the client's Aladdin site	asdk_plugin_platform 2.0.0b3 · UserAPI definition file · line 13
S68	The toolkit is pointed at the client's own Aladdin web server	aladdinsdk 2.0.0b10 · core instructions (README in METADATA) · line 77
S69	Core package release history (author's tally; unverified)	pypi.org · aladdinsdk release history
S70	Add-on release history (Trading shown; all 11 match; author's tally; unverified)	pypi.org · asdk_plugin_trading release history
S71	Release dates as data	pypi.org · aladdinsdk JSON release record

Method and absence checks (method page)

Tag	What it shows	Where
S72	An endpoint that "should no longer be used"	asdk_plugin_portfolio_management 2.0.0b3 · CashLadderAPI definition file · line 46
S73	A cache removal tagged as a read by BlackRock	asdk_plugin_portfolio_management 2.0.0b3 · CIDReaderAPI definition file · line 47
S74	The one use of the word copilot: a portfolio setting about a distribution program	asdk_plugin_portfolio 2.0.0b3 · PortfolioToolkitAPI definition file · line 5713

Outside the package files

Tag	What it shows	Where
S75	What Aladdin is, and the kinds of firms it serves	blackrock.com · Aladdin
S76	Aladdin Data Cloud listed as powered by Snowflake	snowflake.com · Aladdin Data Cloud partner page
S77	Azure API Management is Microsoft's API gateway product	learn.microsoft.com · Azure API Management
S78	pandas, the Python data tool	pandas.pydata.org
S91	Dave Woodhead, head of engineering for Aladdin Studio: three pillars, "3,000 citizen developers," ADC began around 2019, hundreds of Snowflake accounts, AI strategy (quotes from YouTube auto-captions, cleaned)	Snowflake Data Cloud podcast · "From Analytics To Intelligence: BlackRock's Journey To Data Productization" · youtube.com/watch?v=Me_H_-E6lSg
S92	Woodhead at Snowflake Summit 2024: Fivetran plus Snowflake pipelines; clients bring their own data into ADC and distribute Aladdin data out	fivetran.com · "How Fivetran helps power BlackRock's Aladdin Data Cloud," August 28, 2024
S93	Jeff Miller: ADC replaced flat files and FTP	Snowflake · "BlackRock Transforms Client Data Delivery With Snowflake's Aladdin Data Cloud" · youtube.com/watch?v=3iXd7a7_F6g
S94	Richard Newman, GM Aladdin Studio: ADC "is part of Studio"; analytics APIs "releasing soon"	blackrock.com/aladdin/snowflake-audiocast · Never Done podcast, "Data, the key ingredient"
S95	The developer portal's catalog, API-framework docs, Data Dictionary, notebooks and samples; "dozens of APIs"; ADX and ATX listed as Studio resources	blackrock.com/aladdin/products/apis; blackrock.com/aladdin/products/aladdin-studio
S96	Studio for Aladdin Risk: "Run analytics programmatically"; governance "including access management and usage monitoring." Aladdin Copilot "now available to all Aladdin clients"	blackrock.com/aladdin/discover/resources/aladdin-studio-for-aladdin-risk-management; blackrock.com/aladdin/platforms/products/aladdin-copilot
S97	Streaming APIs and webhook subscriptions documented with AsyncAPI; Studio Developer as Aladdin's "extensibility layer"	careers.blackrock.com · Technical Writer, Aladdin Studio; Senior Product Director, Aladdin Studio Developer
S98	Build vs Buy webinar: "you need access to APIs, you need access to events"	blackrock.com/aladdin/discover/expertise/webinars/build-vs-buy-how-asset-managers-are-rethinking-key-decisions
S99	Azure hosting: "all BlackRock's technology platform clients and services are hosted in the Azure cloud"	microsoft.com · Microsoft Cloud Blog, "How BlackRock is finding new alpha," December 11, 2024
S100	AWS hosting option announced December 2025 for H2 2026	press.aboutamazon.com · BlackRock partners with AWS
S101	Auth scheme, scopes, gRPC fingerprint, 190 POST operations, 35 x-aladdin-lro flags, bitemporal fields, four-eyes workflows, 850-attribute Portfolio, BQL fields, Dynaconf, keyring, openapi-generator, api_registry.py and domain_apis_list.json; Copilot tools mapping to existing APIs	All 54 add-on definition files and the core instructions, as recomputed for the companion architecture note; zenml.io summary of BlackRock's 2025 AI engineering talk
S102	Aladdin Data Cloud launched 2021	blackrock.com · press release, "Aladdin Data Cloud powered by Snowflake"
S103	The engineering blog post that open-sourced the SDK for "practitioners and citizen developers"	engineering.blackrock.com · "Open-sourcing the AladdinSDK"

The 12 packages (Exhibit 3 and the method page)

Tag	Package	Where
S79	aladdinsdk 2.0.0b10	pypi.org · aladdinsdk 2.0.0b10
S80	asdk_plugin_accounting 2.0.0b3	pypi.org · asdk_plugin_accounting 2.0.0b3
S81	asdk_plugin_analytics 2.0.0b3	pypi.org · asdk_plugin_analytics 2.0.0b3
S82	asdk_plugin_clients 2.0.0b3	pypi.org · asdk_plugin_clients 2.0.0b3
S83	asdk_plugin_compliance 2.0.0b3	pypi.org · asdk_plugin_compliance 2.0.0b3
S84	asdk_plugin_data 2.0.0b3	pypi.org · asdk_plugin_data 2.0.0b3
S85	asdk_plugin_investment_operations 2.0.0b3	pypi.org · asdk_plugin_investment_operations 2.0.0b3
S86	asdk_plugin_investment_research 2.0.0b3	pypi.org · asdk_plugin_investment_research 2.0.0b3
S87	asdk_plugin_platform 2.0.0b3	pypi.org · asdk_plugin_platform 2.0.0b3
S88	asdk_plugin_portfolio 2.0.0b3	pypi.org · asdk_plugin_portfolio 2.0.0b3
S89	asdk_plugin_portfolio_management 2.0.0b3	pypi.org · asdk_plugin_portfolio_management 2.0.0b3
S90	asdk_plugin_trading 2.0.0b3	pypi.org · asdk_plugin_trading 2.0.0b3

Verification notes for V8

V8 applies a fact-check of V7 against the 11 public add-on packages, BlackRock's product pages, press releases and job postings, and public talks and podcasts. The nine conclusions, the exhibits and the quoted file text are unchanged. What changed:

1. **Framing softened.** V7 said "BlackRock shows little of Studio in public" and that the files are "the closest an outsider can get to seeing what Studio does." Both now refer to the Studio screens and catalog and to the API surface, and the public sources that do exist are named ("What Studio is").
2. **Counts attributed.** The add-on figures (54 APIs, 301 operations; 24 alpha, 6 beta, 23 stable, 1 sunset; the per-package counts in Exhibits 3, 8 and 11; batch, filter, background-job and design-checker counts) were independently recomputed from the public packages and matched V7 exactly. The core-package figures (4 APIs, 25 operations, the demo API) and every total that includes them (58, 326, 27 alpha, 7 beta, 218 scoped, 65 batch, 11 background-job, 90 design-checker notes) are the author's own tally and are marked as such once, under "What the packages contain." The PyPI release history in Exhibit 12 is also the author's own reading; only the 2.0.0b10 version as of July 2026 was independently verified.
3. **Unverified expansions marked.** "ACM stands for Aladdin Credit Management" is now presented as the author's reading of an enum value. "Studio Space … named only in alert settings" now reads "not defined in the files."
4. **Public material added**, each in one short paragraph or a few rows: Woodhead's three-pillar definition of Studio and the 3,000 internal citizen developers with his "probably peaked" caveat; events as a channel in their own right (conclusion 6); the sign-in scheme and the identification of the method section's "access tags" as OAuth scopes; the developer portal's contents, ADX, ATX and the "Aladdin Developer" environment; public support for conclusion 9; Data Cloud's history and scale (conclusion 2) and BlackRock's own "bring their own data" line (conclusion 3); a Copilot note so absence in the files is not read as absence in the product; architecture signals for technical readers (conclusion 8); hosting and people; the DATACLOUD permission group, the three user types in the user API, and the two names for Studio Compute. Every quotation is copied from the cited source; the Woodhead quotes come from cleaned YouTube auto-captions. New sources are S91 to S103.
5. **Format.** V8 is a text-and-table edition. The bar charts and timeline in Exhibits 3, 8 and 12 are rendered as tables with the same figures, and page references are replaced by section references. Exhibit 1's page column is dropped.

About this document

Independent analysis of public package files. It was not produced, reviewed or endorsed by BlackRock.

Facts are as of October 18, 2026, with the V8 revision dated October 19, 2026, unless a table gives another date. The evidence comes from the AladdinSDK core package and the 11 API add-ons published on PyPI, read without being installed or run, and from BlackRock's public pages, press releases, job postings, talks and podcasts. Every source is listed on the sources sheet in the Reference part; the small S tags throughout the report point to it. A black square marks a claim the files state; an orange square marks an inference, rated by confidence.

BlackRock and Aladdin are registered trademarks of BlackRock, Inc. Other company and product names are the property of their owners.

Set in Barlow and Barlow Condensed, open-license stand-ins for BlackRock's licensed BLK Fort typeface.